Predictive: non neural machine learning

Vittorio Maniezzo
Univ. Bologna

Machine learning

Machine learning (ML) is the study of computer algorithms that can improve automatically by the use of data. It is seen as a part of artificial intelligence. Machine learning algorithms build a model based on sample data, known as "training data", in order to make predictions or decisions without being explicitly programmed to do so. *https://en.wikipedia.org/wiki/Machine_learning*

Several ML models can be applied to forecasting: neural networks, but also decision trees, support vector machines (SVMs), Bayesian networks, and others.

Here a hint of SVMs and decision tree learning.

Support Vector Machines

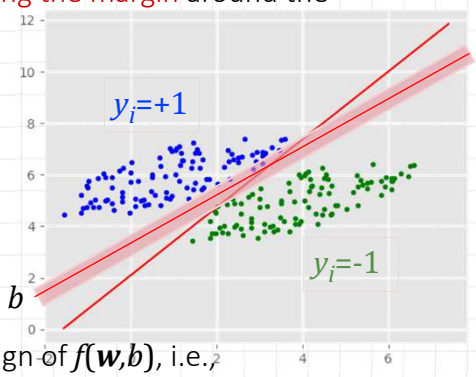
Support Vector Machines are machine learning classifiers which, given labeled training data (supervised learning), compute an optimal hyperplane which separates (categorizes) the examples.

Optimality comes from maximizing the margin around the separating hyperplane.

This increases robustness in classification.

The separating hyperplane is determined by coefficients $(\boldsymbol{w},b)$:

$$f(\boldsymbol{x_i}) = w_1 x_{i1} + \cdots + w_n x_{in} + b$$



The classes are labelled by the sign of $f(\boldsymbol{w},b)$, i.e.,

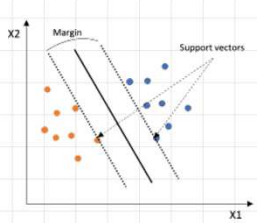
$$y_i = sign\big(f(x_i)\big) \in \{-1,1\}$$

Support Vector Regression

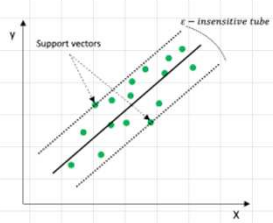
SVR uses hyperplane and margin too, but with differences in their definitions.

The margin, called the ε-insensitive tube, is the error tolerance of the model. This tube allows some deviation of the data points from the hyperplane.

The hyperplane is the best fit possible to the data that fall within the ϵ-insensitive tube.



Classification problem using SVM



Regression problem using SVR

SVR

Support Vector Regression (SVR) with ε-insensitive loss function (Vapnik, 1995) allows a tolerance degree to errors not greater than ε.

SVR gives us through ε the flexibility to define how much error is acceptable in our model and will find an appropriate line (or hyperplane in higher dimensions) to fit the data.

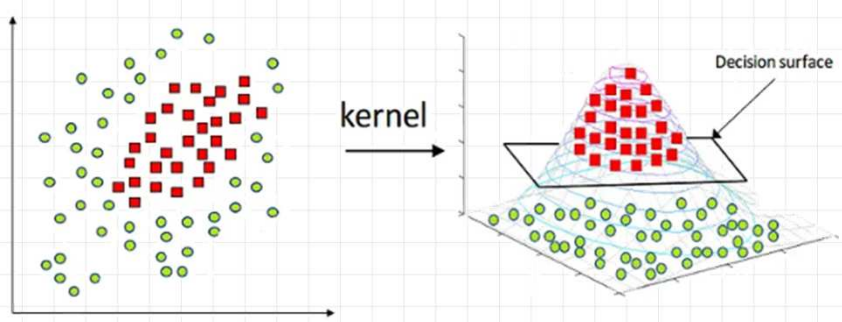
Training data points for the regression model: $\{(x_1,y_1), \dots, (x_\ell,y_\ell)\}$, with $x_i \in \Re^n$ and $y_i \in \Re$.

SVR (*ako* SVM) applies to the input points a transformation function Φ (defining a kernel function) to a higher-dimensional Feature Space F.

In F, we construct a linear model (corresponding to a non-linear model in the input space):

$$\Phi: \mathcal{R}^n \to F, w \in F$$
$$f(x) = \langle w, \Phi(x) \rangle + b$$

SVR, kernel function



SVR

Goal of ε-insensitive loss function is to find a function that *1)* fits training data with deviation less or equal to ε, and *2)* is as flat as possible.

1) Usually impossible, we introduce slack variables ξ_i^-, ξ_i^+ to allow errors greater than ε

2) Obtained minimizing weights, for example their quadratic norm $||w||^2$

SVR

The (primal) problem to solve

min $\frac{1}{2}||w||^2 + C\sum_{i=1}^{t}(\xi_i^- + \xi_i^+)$

s.t. $y_i - (\langle w, \Phi(x_i)\rangle + b) \leq \varepsilon + \xi_i^+$

$\langle w, \Phi(x_i)\rangle + b - y_i \leq \varepsilon + \xi_i^-$

$\xi_i^+, \xi_i^- \geq 0, i = 1, \dots, \ell$

Solving its dual form and projecting to higher dimensional spaces in order to achieve nonlinear separations (Kernel trick) an inner product appears $K(x_i,x_j)=\Phi(x_i)\cdot\Phi(x_j)$, where K is named Kernel function.

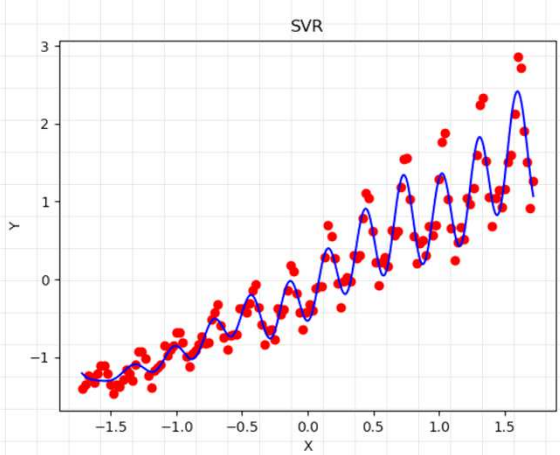
Standard kernels are proposed for actual applications.

SVR in sklearn

```
sc_X = StandardScaler()
sc_y = StandardScaler()
X = X.reshape(-1, 1)
X = sc_X.fit_transform(X)
y = sc_y.fit_transform(Y)

from sklearn.svm import SVR

regressor = SVR(kernel='rbf',C=250.0, gamma=20, epsilon=0.2)
regressor.fit(X, y.flatten())

y_pred = regressor.predict([[6.5]])
y_pred = sc_y.inverse_transform(y_pred)
```


SVR, forecast

Regression models can be directly projected to future periods



Decision trees, ID3

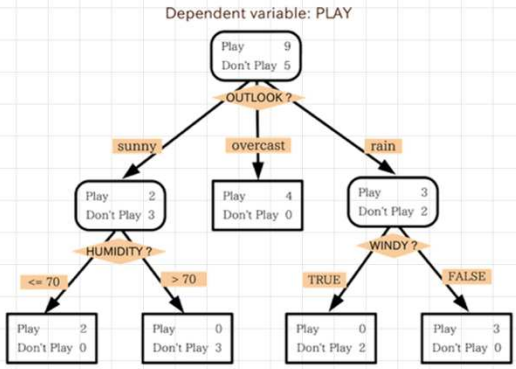
Decision tree: It is a tree-like structure where at every node we make a decision and continue doing it till we reach a conclusion.

The corresponding search algorithm implements a heuristic function using probabilities as comparison values, but does not implement backtracking (greedy).

One of the first popular decision tree algorithms was ID3. Basically, it only constructs a tree data structure and implements two mathematical formula to build complete ID3 algorithm.

Decision tree

Example of a decision tree. In this case, the tree advises us, based upon weather conditions, whether to play ball or not. For example, if the outlook is sunny and the humidity is less than or equal to 70, then it's probably OK to play.



ID3

Steps in ID3 algorithm:

1. It begins with the original set S as the root node.
2. On each iteration of the algorithm, it iterates through every so far unused attribute of the set S and calculates entropy (H) or information gain (IG) of this attribute.
3. It then selects the attribute which has the smallest entropy or largest Information gain.
4. The set S is then split by the selected attribute to produce a subset of the data.
5. The algorithm continues to recur on each subset, considering only attributes never selected before.

ID3: attribute selection

Some criteria are for solving this attribute selection problem:

- Information entropy,
- Information gain (categorical attributes),
- Gini index (continuous attributes),
- Gain Ratio,
- Reduction in Variance
- Chi-Square

These criteria will calculate values for every attribute. The values are sorted, and attributes are placed in the tree by following the order.

Information entropy

IE is a fundamental quantity commonly used in information theory to measure importance of information relative to its size.

If X is the training set containing positive and negative examples, then the entropy of X relative to this classification is:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

ID3 follows the rule — A branch with an entropy of zero is a leaf node and a branch with entropy more than zero needs further splitting.

Play Golf	
Yes	No
9	5

Entropy(PlayGolf) = Entropy (5,9)
= Entropy (0.36, 0.64)
= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)
= 0.94

Information gain

IG is a statistical property that measures how well a given attribute separates the training examples according to their target classification. Information gain is a decrease in entropy. It computes the difference between entropy before split and average entropy after split of the dataset based on given attribute values.

For training set x and its attribute y, the formula of Information Gain is $G(x,y) = H(x) - \sum_{i \in value(y)} \frac{|\Delta y_i|}{|\Delta y|} H(y_i)$

Constructing a decision tree requires finding an attribute that returns the highest information gain and the smallest entropy.

ID3, pseudocode

```
# x is the set of examples in training set, y is the set of attributes
# labels are labeled data
# Node is a class which has properties values, childs, and next
# root is top node in the decision tree
x = # Multi dimensional arrays
y = # Column names of x
labels = # Classification values, for example {0, 1, 0, 1}
         # correspond that row 1 is false, row 2 is true, and so on
root = ID3(x, y, label, root)

def ID3(x, y, label, node)
  initialize node as a new node instance
  if all rows in x only have single classification c, then:
    insert label c into node
    return node
  if x is empty, then:
    insert dominant label in x into node
    return node
  bestAttr is an attribute with maximum information gain in x
  insert attribute bestAttr into node
  for vi in values of bestAttr:
    # For example, Outlook has three values: Sunny, Overcast, and Rain
    insert value vi as branch of node
    create viRows with rows that only contains value vi
    if viRows is empty, then:
      this node branch ended by a leaf with value is dominant label in x
    else:
      newY = list of attributes y with bestAttr removed
      nextNode = next node connected by this branch
      nextNode = ID3(viRows, newY, label, nextNode)
  return node
```

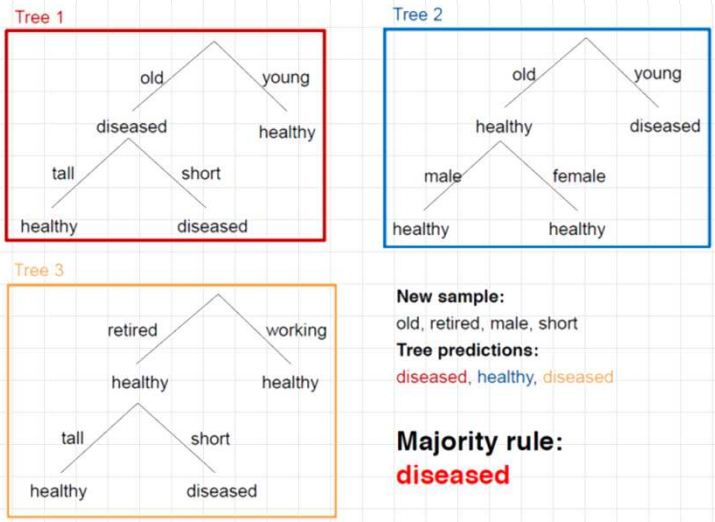

Building Ensemble Classifiers

- Basic idea: Build different classifiers, and let them vote
- Advantages:
  - Improve predictive performance
  - Other types of classifiers can be directly included
  - Easy to implement
  - Not too much parameter tuning
- Disadvantages:
  - The combined classifier is not so transparent (black box)
  - Not a compact representation.

Boosting: Training a bunch of models sequentially. Each model *learns from the mistakes* of the previous model. That is, the subsequent models tries to explain and predict the error left over by the previous model.

Bagging: Training a bunch of models in parallel. Each model learns from a *random subset of the data*, where the dataset is same size as original but is randomly sampled with replacement (bootstrapped).

Tree ensembles



New sample:
old, retired, male, short
Tree predictions:
diseased, healthy, diseased

Majority rule:
diseased

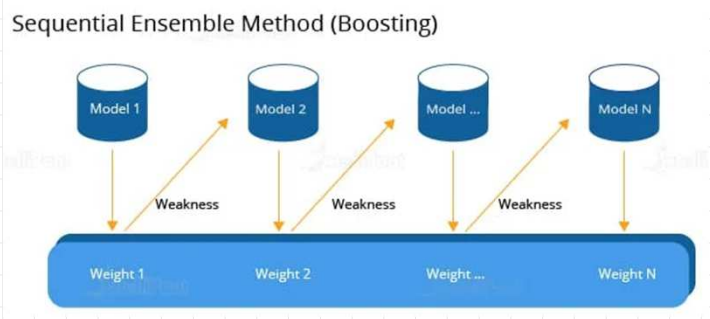
Why do they work?

- Suppose there are 25 base classifiers
- Each classifier has error rate ε = 0.35
- Assume independence among classifiers
- Probability that the ensemble classifier makes a wrong prediction (binomial):

$$\sum_{i=13}^{25} \binom{25}{i} \varepsilon^{i} (1-\varepsilon)^{25-i} = 0.06$$

Boosting (sequential)

In boosting we train the individual models in a sequential way. Each individual model learns from mistakes made by the previous model. AdaBoost, Gradient Boost etc. are different types of boosting methods.



Gradient Boosting Decision Trees

In gradient boosting decision trees, we combine many weak learners to come up with one strong learner. The weak learners here are the individual decision trees.

All the trees are connected in series and each tree tries to minimize the error of the previous tree.

Due to this sequential connection, boosting algorithms are usually slow to learn, but also highly accurate.

The weak learners are fit in such a way that each new learner fits into the residuals of the previous step so that the model improves.

Boosting

Originally developed by computational learning theorists to guarantee performance improvements on fitting training data for a ***weak learner*** that only needs to generate a hypothesis with a training accuracy greater than 0.5.

Revised to be a practical algorithm, AdaBoost, for building ensembles that empirically improves generalization performance.

Improvement, weighting:

- Instead of sampling re-weigh examples
- Examples are given weights. At each iteration, a new hypothesis is learned (weak learner) and the examples are reweighted to focus the system on examples that the most recently learned classifier got wrong.
- Final classification based on weighted vote of weak classifiers

Construct Weak Classifiers

Using Different Data Distribution

- Start with uniform weighting
- During each step of learning
  - Increase weights of the examples which are not correctly learned by the weak learner
  - Decrease weights of the examples which are correctly learned by the weak learner

Combine Weak Classifiers

Weighted Voting

- Construct strong classifier by weighted voting of the weak classifiers

Idea

- Better weak classifier gets a larger weight
- Iteratively add weak classifiers
  - Increase accuracy of the combined classifier through minimization of a cost function

Gradient boosting

Boosting is a numerical optimization problem where the objective is to minimize the loss of the model by adding weak learners using a gradient descent like procedure.

Algorithms using boosting are described as stage-wise additive models: a new weak learner is added at a time and existing weak learners in the model are frozen and left unchanged.

Gradient boosting involves three elements:

- A loss function to be optimized.
- A weak learner to make predictions.
- An additive model to add weak learners to minimize the loss function.

Gradient boosting

Decision trees are used as the weak learner in gradient boosting.

Specifically, regression trees are used that output real values for splits and whose output can be added together, allowing subsequent models outputs to be added and correct the residuals in the predictions.

Trees are constructed in a greedy manner, choosing the best split points based on purity scores like Gini or to minimize the loss.

Initially, f.e. in AdaBoost, very short decision trees are used that only had a single split, called a decision stump. Larger trees with 4-to-8 levels can later be used.

It is common to constrain the weak learners in specific ways, such as a maximum number of layers, nodes, splits or leaf nodes.

Gradient boosting

Additive Model: trees are added one at a time, and existing trees in the model are not changed (greedy).

A gradient descent procedure is used to minimize the loss when adding trees.

Instead of optimizing parameters, we add more specifical decision trees. To perform the gradient descent, we add a tree to the model that most reduces the loss (i.e. we follow the gradient). We do this by parameterizing the tree, then modifying the parameters of the tree and move in the right direction by reducing the residual loss.

The output for the new tree is then added to the output of the existing sequence of trees in an effort to correct or improve the final output of the model.

A fixed number of trees are added or training stops once loss reaches an acceptable level or no longer improves on an external validation dataset.

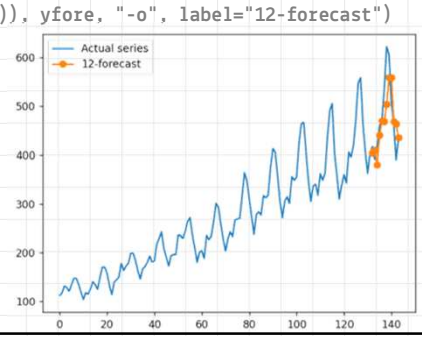
xGBoost

The XGBoost library has its own Python API, but we can also use XGBoost models with the scikit-learn syntax via the XGBRegressor wrapper class.

```
from xgboost import XGBRegressor
# rolling window dataset (see MLP)
df = pd.read_csv('BoxJenkins.csv', usecols=[1], names=['Passengers'], header=0)

# rolling window dataset (see MLP)
lookback = 12
X,y = create_dataset(df["Passengers"].values,lookback)
nfore = 12
x_train, _     = X[:-nfore], X[-nfore:]
y_train, ytest = y[:-nfore], y[-nfore:]

# fit model
model = XGBRegressor(objective='reg:squarederror', n_estimators=1000)
model.fit(x_train, y_train)
```


xGBoost

```
# rolling 12 step prediction
  xinput = x_train[-1]
  yfore  = []
  for i in range(12):
     yfore.append(model.predict(xinput.reshape(1,12))[0])
     xinput = np.roll(xinput,-1)
     xinput[-1] = yfore[-1]

  plt.plot(df.values,label="Actual series")
  plt.plot(range(len(df)-nfore,len(df)), yfore, "-o", label="12-forecast")
  plt.legend()
  plt.show()
```



Bagging (parallel)

Bagging: decision trees are sensitive to the specific data on which they are trained. If the training data is changed (e.g. a tree is trained on a subset of the training data) the resulting decision tree and the predictions can be quite different.

To overcome this problem, we create N learners (decision trees) and produce N new training data sets by random sampling with replacement from the original set. The final prediction is the average of prediction from N decision trees.

Random Forest is a bagging method. It adds an extra step. Along with creating N learners that use a randomly sampled training data set, it also does a random selection of features rather than using all features to grow trees.

Bagging, Aggregate Bootstrapping

Given a standard training set D of size n

For i = 1 .. M

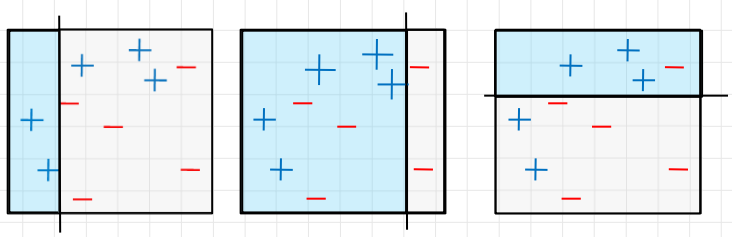
- Draw a sample of size $n^* \leq n$ from D uniformly and with replacement
- Learn classifier C_i

Final classifier derived from a vote by $C_1 .. C_M$

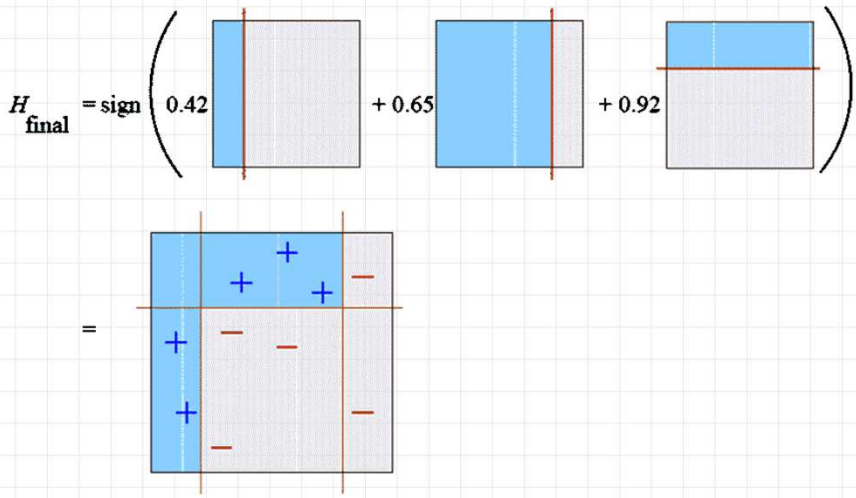
Bagging

- Training:
  - Given a dataset S, at each iteration i, a training set S_i is sampled with replacement from S (i.e. bootstraping)
  - A classifier C_i is learned for each S_i
- Classification: given an unseen sample X,
  - Each classifier C_i returns its class prediction
  - The bagged classifier H counts the votes and assigns the class with the most votes to X
- Regression: can be applied to the prediction of continuous values by taking the average value of each prediction.

Toy Example



Final Classifier



35

Random forest

Random forest is an ensemble of decision tree algorithms.

It is an extension of bootstrap aggregation (bagging) of decision trees and can be used both for classification and regression problems.

Each decision tree is fit on a slightly different training dataset, and in turn, has a slightly different performance.

Unlike normal decision tree models, trees used in the ensemble are unpruned, making them slightly overfit to the training dataset. This is desirable as it helps to make each tree more different and have less correlated predictions or prediction errors.

36

Random forest

Predictions from the trees are averaged across all decision trees, resulting in better performance than any single tree in the model.

Unlike bagging, random forest also involves selecting a subset of input features (columns or variables) at each split point in the construction of the trees.

A prediction on a regression problem is the average of the prediction across the trees in the ensemble.

A prediction on a classification problem is the majority vote for the class label across the trees in the ensemble.

Typically, constructing a decision tree involves evaluating the value for each input variable in the data in order to select a split point. By reducing the features to a random subset that may be considered at each split point, it forces each decision tree in the ensemble to be more different.

The effect is that the predictions (and the prediction errors), made by each tree are more different and less correlated. When the predictions from these less correlated trees are averaged, it often results in better performance than bagged decision trees.

How Random Forest Work

Each tree is grown on a bootstrap sample of the training set of n cases.

A number m is specified much smaller than the total number of variables M (e.g., $m = \mathrm{sqrt}(\mathrm{M})$).

At each node, m variables are selected at random out of the M.

The split used is the best split on these m variables.

Final classification is done by majority vote across trees.

Training the algorithm

For some number of trees *T*:

Sample *N* cases at random with replacement to create a subset of the data. The subset should be about 66% of the total set.

At each node:

- For some number *m*, *m* predictor variables are selected at random from all the predictor variables.
- The predictor variable that provides the best split, according to some objective function, is used to do a binary split on that node.
- At the next node, choose another *m* variables at random from all predictor variables and do the same.

Random forest: should be $m << \textit{number of predictor variables}$.

Values suggested for m : $\frac{1}{2}\sqrt{m}, \sqrt{m}, 2\sqrt{m},$

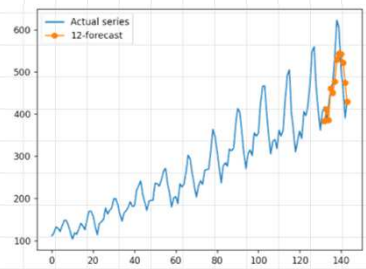
Random forest

Airline passengers example, train/test sets as for XGboost

```
# random forest for making predictions for regression
from sklearn.ensemble import RandomForestRegressor
from sklearn.feature_selection import RFE
from sklearn.metrics import mean_absolute_error

# rolling window dataset (see MLP)
df = pd.read_csv('BoxJenkins.csv', usecols=[1], names=['Passengers'], header=0)

# rolling window dataset (see MLP)
lookback = 12
X,y = create_dataset(df["Passengers"].values,lookback)
nfore = 12
x_train, _ = X[:-nfore], X[-nfore:]
y_train, ytest = y[:-nfore], y[-nfore:]

# define the model
RFmodel = RandomForestRegressor(n_estimators=500, random_state=1)
RFmodel.fit(x_train, y_train)
```

Random forest

```
# rolling 12 step prediction
xinput = x_train[-1]
yfore  = []
for i in range(12):
    yfore.append(RFmodel.predict(xinput.reshape(1,12))[0])
    xinput = np.roll(xinput,-1)
    xinput[-1] = yfore[-1]
```

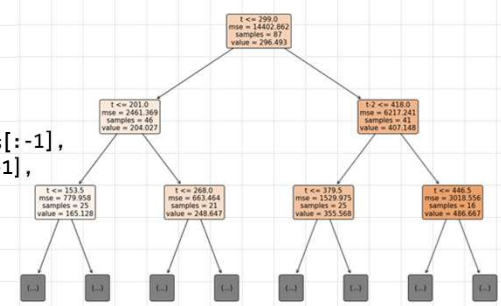



```
mse = mean_absolute_error(ytest, yfore)
print("MSE={}".format(mse))

plt.plot(df.values,label="Actual series")
plt.plot(range(len(df)-nfore,len(df)),yfore,"-o",label="12-forecast")
plt.legend()
plt.show()
```


41

Random forest, estimators

```
# Stats about the trees in random forest
n_nodes = []
max_depths = []
for ind_tree in RFmodel.estimators_:
    n_nodes.append(ind_tree.tree_.node_count)
    max_depths.append(ind_tree.tree_.max_depth)
print(f'Average number of nodes {int(np.mean(n_nodes))}')
print(f'Average maximum depth {int(np.mean(max_depths))}')

# plot first tree (index 0)
from sklearn.tree import plot_tree
fig = plt.figure(figsize=(15, 10))
plot_tree(RFmodel.estimators_[0],
          max_depth=2,
          feature_names=dataset.columns[:-1],
          class_names=dataset.columns[-1],
          filled=True, impurity=True,
          rounded=True)
plt.show()
```



42

Feature selection

```
# find the columns with most predictive power
rfe = RFE(RFmodel, n_features_to_select=4) # Recursive Feature
Elimination, a feature selection algorithm
fit = rfe.fit(X, y)
names = featNames
predictors = []
for i in range(len(fit.support_)):
   if fit.support_[i]:
      predictors.append(names[i])
print("Columns with predictive power:", predictors)
```


Compared with Boosting

Pros:

- It is more robust.
- It is faster to train (no reweighting, each split is on a small subset of data and features).
- Can handle missing/partial data.
- Is easier to extend to online version.

Cons:

- The feature selection process is not explicit.
- Feature fusion is also less obvious.
- Has weaker performance on small size training data.